

Spring Boot

CatMono

March 23, 2026

目 录

I	Spring Boot 基础	
1 第 01 章		3
1 占位内容		3
2 第 02 章		4
1 占位内容		4
3 第 03 章		5
1 占位内容		5
4 第 04 章		6
1 占位内容		6
II	Spring Boot 核心	
5 第 05 章		8
1 占位内容		8
6 第 06 章		9
1 占位内容		9
7 第 07 章		10
1 占位内容		10
8 第 08 章		11
1 占位内容		11
9 第 09 章		12
1 占位内容		12
10 第 10 章		13

1	占位内容	13
11	第10章	14
1	占位内容	14

III

微服务与中间件

12	微服务架构基础	16
1	微服务架构概述	16
1.1	微服务架构简介	16
1.2	Spring Cloud 与微服务生态	17
2	服务注册与发现	19
2.1	核心组件	19
2.2	主流实现	20
3	配置中心	20
4	API 网关	20
5	服务调用	20
6	负载均衡与熔断降级	20
13	第10章	21
1	占位内容	21
14	第10章	22
1	占位内容	22

IV

Spring Boot 源码

I

Part

Spring Boot 基础

1	第 01 章	3
1	占位内容	3
2	第 02 章	4
1	占位内容	4
3	第 03 章	5
1	占位内容	5
4	第 04 章	6
1	占位内容	6

Chapter 1

第 01 章

1 占位内容

这里是 chap01 的初始内容。

Chapter 2

第 02 章

1 占位内容

这里是 chap02 的初始内容。

Chapter 3

第 03 章

1 占位内容

这里是 chap03 的初始内容。

Chapter 4

第 04 章

1 占位内容

这里是 chap04 的初始内容。

II

Part

Spring Boot 核心

5	第 05 章	8
1	占位内容	8
6	第 06 章	9
1	占位内容	9
7	第 07 章	10
1	占位内容	10
8	第 08 章	11
1	占位内容	11
9	第 09 章	12
1	占位内容	12
10	第 10 章	13
1	占位内容	13
11	第 10 章	14
1	占位内容	14

Chapter 5

第 05 章

1 占位内容

这里是 chap05 的初始内容。

Chapter 6

第 06 章

1 占位内容

这里是 chap06 的初始内容。

Chapter 7

第 07 章

1 占位内容

这里是 chap07 的初始内容。

Chapter 8

第 08 章

1 占位内容

这里是 chap08 的初始内容。

Chapter 9

第 09 章

1 占位内容

这里是 chap09 的初始内容。

Chapter 10

第 10 章

1 占位内容

这里是 chap10 的初始内容。

Chapter 11

第 10 章

1 占位内容

这里是 chap10 的初始内容。

Part III

微服务与中间件

12	微服务架构基础	16
1	微服务架构概述	16
2	服务注册与发现	19
3	配置中心	20
4	API 网关	20
5	服务调用	20
6	负载均衡与熔断降级	20
13	第 10 章	21
1	占位内容	21
14	第 10 章	22
1	占位内容	22

Chapter 12

微服务架构基础

1 微服务架构概述

1.1 微服务架构简介

微服务 (Microservices) 是一种软件架构风格, 它是以专注于单一责任与功能的小型功能区块 (Small Building Blocks) 为基础, 利用模块化的方式组合出复杂的大型应用程序, 各功能区块使用与语言无关 (Language Independent), 而且复杂的服务背后是使用简单 URI 来开放界面, 任何服务, 任何细粒都能被开放 (exposed). 这个设计在 HP 的实验室被实现, 具有改变复杂软件系统的强大力量。

2014 年, Martin Fowler 与 James Lewis 共同提出了微服务的概念, 定义了微服务是由以单一应用程序构成的小服务, 自己拥有自己的进程与轻量化处理, 服务依业务功能设计, 以全自动的方式部署, 与其他服务使用 HTTP API 通信。同时服务会使用最小的规模的集中管理 (例如 Docker) 能力, 服务可以用不同的编程语言与数据库等组件实现。

核心理念: “分而治之”, 将单体应用按业务领域 (Domain-Driven Design, DDD) 拆分为多个松耦合的服务。核心特征:

单一职责 每个微服务专注于一个业务功能, 职责单一, 易于理解和维护。

独立部署 每个微服务可以独立构建、测试和部署, 减少了部署风险和复杂度。

技术异构 不同微服务可以使用不同的编程语言、数据库和技术栈, 适合不同的业务需求。

轻量通信 微服务之间通过轻量级协议 (如 HTTP/REST、gRPC) 进行通信, 降低了服务间的耦合度。

自治性 每个微服务拥有自己的数据库和数据管理方式, 避免了共享数据库带来的耦合问题。

弹性与可伸缩性 微服务可以根据需求独立伸缩, 提高系统的弹性和性能。

下表 (12.1) 是单体应用、SOA 和微服务架构的对比:

NOTE

ESB (Enterprise Service Bus) 是一种**重量级**的企业级服务总线, 提供了服务注册、发现、路由、协议转换等功能, 适用于 SOA 架构。

SOAP (Simple Object Access Protocol) 是一种基于 XML 的协议, 用于在网络上交换结构化信息, 常用于 SOA 架构中的服务通信。

gRPC (Google Remote Procedure Call) 是一种高性能、开源的远程过程调用框架, 支持多种编程语言, 常用于微服务架构中的服务通信。

微服务架构在带来灵活性、可伸缩性和快速迭代的同时, 也引入了分布式系统的复杂性, 如服务间通信、数据一致性、监控和调试、运维复杂性等挑战。因此, 选择微服务架构需要根据具体业务需求、

Table 12.1: 单体应用、SOA 与微服务架构对比

对比维度	单体应用	SOA	微服务
服务粒度	粗粒度，模块集中在同一应用中	中等粒度，按企业能力划分服务	细粒度，按业务能力拆分为小服务
部署方式	整体打包与统一部署	服务可独立部署，但常依赖集中式平台	服务独立构建、独立部署、可独立扩缩容
通信方式	进程内调用为主	常见 ESB、SOAP 等较重通信方式	轻量协议（HTTP/REST、gRPC、消息队列）
数据管理	通常共享一个数据库	可能存在共享数据模型	每个服务拥有自治数据，避免强耦合
技术栈	技术栈统一，升级牵一发动全身	相对统一，强调企业级标准化	技术异构，按场景选择最合适技术
适用场景	业务简单、团队小、快速起步	跨系统集成、存量系统较多的大型企业	业务复杂、迭代快、需要高可用与弹性扩展

团队能力和技术栈进行权衡。

1.2 Spring Cloud 与微服务生态

Spring Cloud 是 Pivotal 团队提供的开源微服务框架，是基于 Spring Boot 的一套工具集，它将 Netflix、Alibaba 等公司的成熟微服务组件整合到 Spring 生态中，帮助开发者快速构建微服务架构。Spring Cloud 提供了以下核心组件：

服务注册与发现 Nacos、Consul、Eureka (已停止维护) 等组件提供了服务注册与发现功能，支持服务实例的动态注册和查询。Nacos 集成了注册与配置中心，成为事实标准。

配置中心 Nacos Config、Spring Cloud Config 等组件提供了集中化的配置管理，支持动态刷新和版本控制。

API 网关 Spring Cloud Gateway 提供了一个基于 Spring WebFlux 的非阻塞网关，支持路由、负载均衡、熔断、限流等功能。

服务调用 Spring Cloud OpenFeign 提供了声明式的 REST 客户端，简化了服务间的调用和通信。

负载均衡与熔断降级 Spring Cloud LoadBalancer 提供了客户端负载均衡功能，Resilience4j 提供了熔断和降级功能，帮助提高系统的稳定性和容错能力。

INFO

Spring Cloud 2020.x 之后移除了 Netflix 组件 (Ribbon、Hystrix、Zuul 1.x)，转向 Spring Cloud LoadBalancer、Resilience4j、Spring Cloud Gateway。

2025 年起，Nacos 与 Kubernetes 原生服务发现成为主流，Spring Cloud Kubernetes 提供了与 K8s 原生服务注册的集成。

Spring Cloud 的版本命名早期使用 London 地铁站名 (如 Hoxton、Greenwich)，2020 年后使用日历化版本 (如 2020.0.x、2021.0.x)。其版本对应关系如表 (12.2, 12.3, 12.4) 所示。

WARNING

Spring 官方说明：Dalston、Edgware、Finchley、Greenwich、2020.0、2021.0、2022.0 已结束支持 (EOL)。

新项目开发建议使用 SpringBoot 3.2.4 + Spring Cloud 2023.0.1 + Spring Cloud Alibaba

Table 12.2: Spring Cloud Release Train 与 Spring Boot 对应关系（官方）

Release Train	Spring Boot Generation
2025.1.x (Oakwood)	4.0.x
2025.0.x (Northfields)	3.5.x
2024.0.x (Moorgate)	3.4.x
2023.0.x (Leyton)	3.3.x, 3.2.x
2022.0.x (Kilburn)	3.0.x, 3.1.x (Starting with 2022.0.3)
2021.0.x (Jubilee)	2.6.x, 2.7.x (Starting with 2021.0.3)
2020.0.x (Ilford)	2.4.x, 2.5.x (Starting with 2020.0.3)
Hoxton	2.2.x, 2.3.x (Starting with SR5)
Greenwich	2.1.x
Finchley	2.0.x
Edgware	1.5.x
Dalston	1.5.x

Table 12.3: Spring Cloud Alibaba 2025.x 2021.x 版本对应关系（官方）

Spring Cloud Alibaba Version	Spring Cloud Version	Spring Boot Version
2025.1.0.0	2025.1.0	4.0.0
2025.0.0.0	2025.0.0	3.5.0
2023.0.1.0	Spring Cloud 2023.0.1	3.2.4
2023.0.0.0-RC1	Spring Cloud 2023.0.0	3.2.0
2022.0.0.0	Spring Cloud 2022.0.0	3.0.2
2022.0.0.0-RC2	Spring Cloud 2022.0.0	3.0.2
2022.0.0.0-RC1	Spring Cloud 2022.0.0	3.0.0
2021.0.6.0	Spring Cloud 2021.0.5	2.6.13
2021.0.5.0	Spring Cloud 2021.0.5	2.6.13
2021.0.4.0	Spring Cloud 2021.0.4	2.6.11
2021.0.1.0	Spring Cloud 2021.0.1	2.6.3
2021.1	Spring Cloud 2020.0.1	2.4.2

Table 12.4: Spring Cloud Alibaba 2.2.x 及以下版本对应关系（官方）

Spring Cloud Alibaba Version	Spring Cloud Version	Spring Boot Version
2.2.10-RC2*	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.10-RC1	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.9.RELEASE	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.8.RELEASE	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.7.RELEASE	Spring Cloud Hoxton.SR12	2.3.12.RELEASE
2.2.6.RELEASE	Spring Cloud Hoxton.SR9	2.3.2.RELEASE
2.2.1.RELEASE	Spring Cloud Hoxton.SR3	2.2.5.RELEASE
2.2.0.RELEASE	Spring Cloud Hoxton.RELEASE	2.2.X.RELEASE
2.1.4.RELEASE	Spring Cloud Greenwich.SR6	2.1.13.RELEASE
2.1.2.RELEASE	Spring Cloud Greenwich	2.1.X.RELEASE
2.0.4.RELEASE（停止维护，建议升级）	Spring Cloud Finchley	2.0.X.RELEASE
1.5.1.RELEASE（停止维护，建议升级）	Spring Cloud Edgware	1.5.X.RELEASE

2023.0.1.0 版本组合。SpringBoot 3.2.4 版本是 3.2 系列的稳定补丁版，配套 Spring Framework 6.1.x、Spring Security 6.2.x，最低支持 Java 17，推荐使用 Java 21。

2 服务注册与发现

在微服务架构中，一个完整的应用被拆分成多个独立的服务。这些服务通常运行在不同的服务器上，拥有动态的 IP 地址和端口号，且服务实例的位置是动态变化的（扩缩容、故障重启、发布更新），尤其是在使用容器化技术（如 Docker）和云平台时。

这就带来了一个核心问题：服务 A 如何知道服务 B 的地址并与之通信？换句话说，[服务消费者如何找到服务提供者](#)？

- 传统单体应用：不同模块之间通过方法调用直接通信，地址是固定的。
- 微服务应用：服务地址是动态变化的。如果硬编码 IP 地址和端口，一旦服务重启、扩容或缩容，地址就会失效，导致整个系统瘫痪。

[服务注册与发现机制](#) 就是为了解决这个问题而诞生的。它提供了一个动态、自动化的方式来管理服务间的通信地址。

2.1 核心组件

服务注册与发现模式主要包含三个角色：

1. 服务注册中心 (Service Registry): 一个集中式的服务目录，相当于一个“微服务通讯录”，负责维护所有可用服务实例的信息，包括服务名称、IP 地址、端口号等元数据。
2. 服务提供者 (Service Provider): 提供具体业务功能的服务实例，在启动时向服务注册中心**注册**自己的信息，并定期发送心跳 (Heartbeat) 以保持注册状态。
3. 服务消费者 (Service Consumer): 需要调用其他服务的服务实例，在调用时向服务注册中心**查询**目标服务的地址列表，注册中心返回一个或多个地址。消费者会选择其中一个（通常通过负载均衡策略）发起调用。



Figure 12.1: 服务注册与发现机制示意图

2.2 主流实现

组件	来源	特点	状态与推荐度
Eureka	Netflix	- 经典的注册中心，易于使用 - AP 架构（高可用性优先） - 服务端和客户端模式 - P2P 架构，所有节点平等 - 维护模式（Maintenance Mode）。不推荐在新项目中使用，但仍广泛存在于老项目中。	
Nacos	Alibaba	- 功能强大：集服务注册中心和配置中心于一体 - CP/AP 架构可切换 - 支持服务权重、健康检查、元数据管理 - 提供可视化控制台	强烈推荐。功能全面，社区活跃，是目前国内企业的主流选择。
Consul	HashiCorp	- 功能强大，生态完整 - CP 架构（强一致性优先） - 提供 KV 存储、多数据中心支持 - 健康检查机制非常强大	推荐。尤其适合对一致性要求高、或已在使用 HashiCorp 其他产品的技术栈。
Zookeeper	Apache	- 分布式协调服务，也可作为注册中心 - CP 架构（强一致性） - 树形节点数据结构	较老，配置相对复杂，在新微服务项目中较少作为首选注册中心。

- 3 配置中心
- 4 API 网关
- 5 服务调用
- 6 负载均衡与熔断降级

Chapter 13

第 10 章

1 占位内容

这里是 chap10 的初始内容。

Chapter 14

第 10 章

1 占位内容

这里是 chap10 的初始内容。

IV

Part

Spring Boot 源码